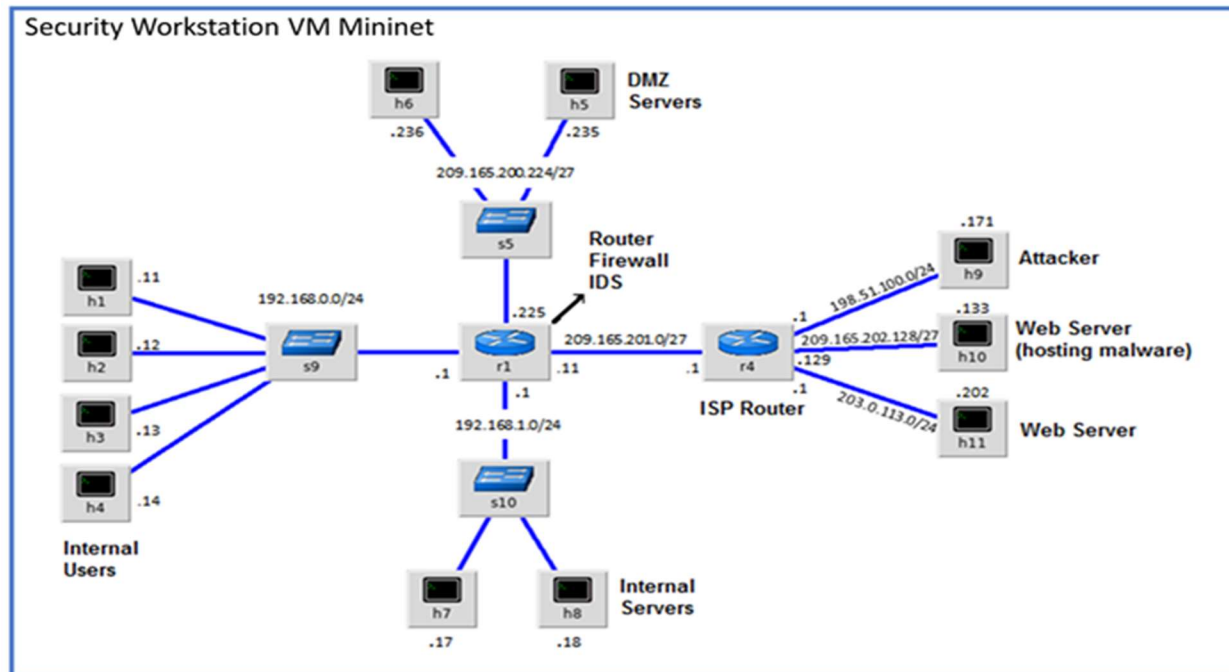


Portfolio Project #01

Snort IDS and Firewall Rules Lab

Executive Summary



This project documents the successful completion of the Snort and Firewall Rules lab from the Cisco Networking Academy Network Defense course. The lab simulated the workflow of a Security Operations Center (SOC) analyst by demonstrating how an Intrusion Detection System (IDS) can detect malicious activity and how firewall rules can be implemented to prevent similar attacks from recurring.

Using the Cisco Security Workstation virtual machine running in Oracle VirtualBox, I configured the virtual network environment, launched a simulated enterprise topology using Mininet, monitored network traffic with Snort, captured malicious traffic using tcpdump, and implemented firewall rules using iptables.

A simulated malware download successfully triggered a Snort alert, demonstrating how signature-based intrusion detection identifies malicious traffic. After analyzing the alert, firewall rules were configured to block further downloads of the malicious file. The effectiveness of the mitigation was verified by confirming that subsequent download attempts were denied.

This lab strengthened my understanding of intrusion detection, firewall configuration, packet capture, and the practical workflow followed by SOC analysts during threat detection and containment.

1. Introduction

Cybersecurity professionals use multiple layers of defense to detect and prevent attacks. Two important security technologies are Intrusion Detection Systems (IDS) and firewalls. While an IDS monitors network traffic and generates alerts when suspicious activity is detected, a firewall enforces security policies by allowing or blocking network traffic.

This lab provided practical experience configuring and using Snort as a Network Intrusion Detection System (NIDS). It also demonstrated how firewall rules created with iptables can prevent malicious network activity after it has been identified. The exercise simulated a real-world security monitoring scenario where malicious activity is detected, investigated, and contained.

2. Objectives

- Understand the role of Snort as a Network Intrusion Detection System (NIDS).
- Monitor and analyze network traffic for malicious activity.
- Launch and manage a simulated enterprise network using Mininet.
- Capture network traffic using tcpdump.
- Interpret Snort alerts generated by malicious activity.
- Configure firewall rules using iptables.
- Verify that firewall rules successfully block malicious traffic.
- Understand the purpose of the INPUT, OUTPUT, and FORWARD chains in iptables.
- Experience a workflow similar to that of a SOC analyst by detecting, investigating, and containing a simulated attack.

3. Lab Environment

Component	Description
Virtualization Software	Oracle VirtualBox
Virtual Machine	Cisco Security Workstation VM
Operating System	Linux (Cisco Security Workstation)
Network Emulator	Mininet

Intrusion Detection System	Snort
Packet Capture Tool	tcpdump
Firewall	iptables
Protocols Used	ICMP, HTTP

```

Node: H5 (as superuser)
[root@secOps analyst]#

Node: H10 (as superuser)
secOps analyst#

Node: R1 (as superuser)
Using ZLIB version: 1.2.11
...
208.112): icmp_seq=15 ttl=41 time=173 ms
on a23-214-208-112.deploy.static.akamaitechnologies.com (23.21
cnp_seq=16 ttl=41 time=562 ms
on a23-214-208-112.deploy.static.akamaitechnologies.com (23.21
cnp_seq=17 ttl=41 time=278 ms
on a23-214-208-112.deploy.static.akamaitechnologies.com (23.21
cnp_seq=18 ttl=41 time=197 ms
on a23-214-208-112.deploy.static.akamaitechnologies.com (23.21
cnp_seq=19 ttl=41 time=149 ms

sca.akamaiedge.net ping statistics ---
transmitted, 19 received, 0% packet loss, time 22475ms
/max/ndev = 140.595/247.542/562.392/93.493 ms
cOps ~]$ sudo ./lab.support.files/scripts/cyberops_extended_to

controller
tches
ts
ks
g network
ring hosts
H3 H4 H5 H6 H7 H8 H9 H10 H11
g controllers
g switches
tes
nfigure switches and hosts
g CLI:
erm R1
erm H5
.net> xterm H10
.net>
10.101.01011

```

4. Methodology

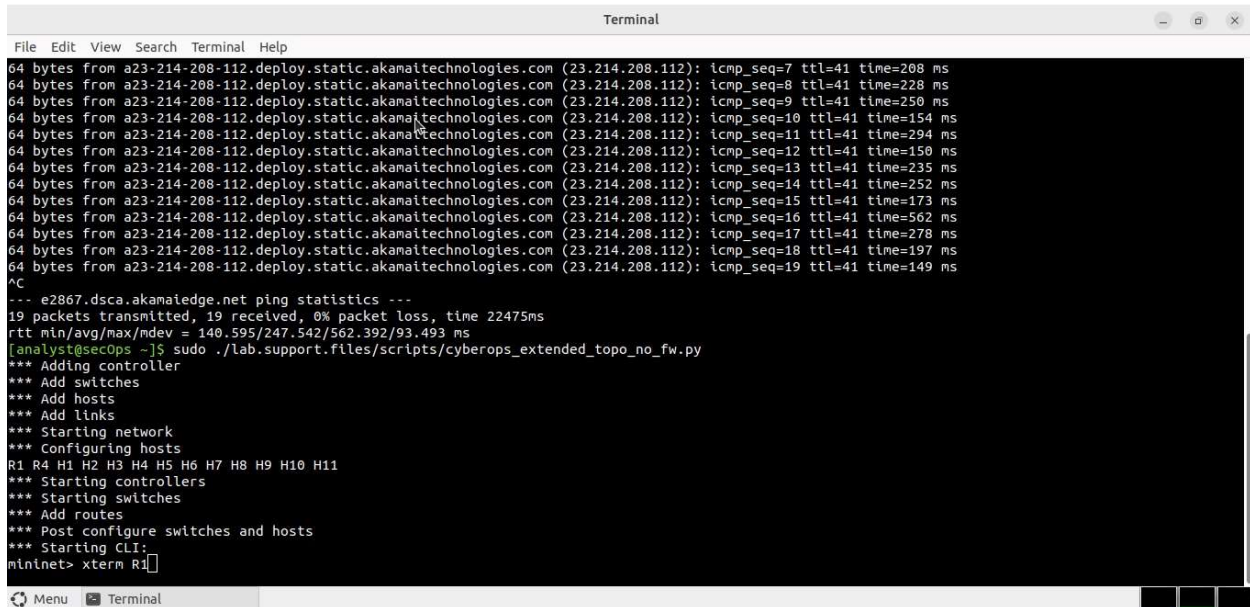
The first step was to verify network connectivity by successfully pinging Cisco's website. The workstation was then configured to obtain network settings automatically using DHCP, and the network configuration was verified.

```

Terminal
File Edit View Search Terminal Help
[analyst@secOps ~]$ sudo ./lab.support.files/scripts/configure_as_dhcp.sh
[sudo] password for analyst:
Configuring the NIC to request IP info via DHCP...
Requesting IP information...
IP Configuration successful.
[analyst@secOps ~]$ 

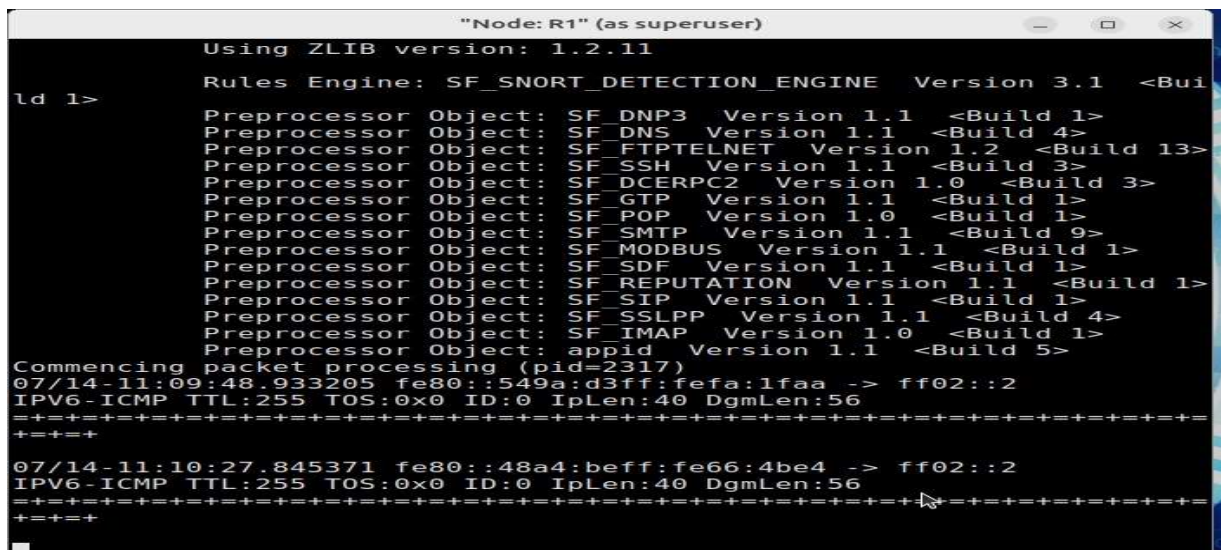
```

The Mininet enterprise topology was launched using the provided Cisco topology script. Separate terminal windows were opened for the simulated router (R1) and hosts (H5 and H10), allowing multiple network components to be managed simultaneously.



```
Terminal
File Edit View Search Terminal Help
64 bytes from a23-214-208-112.deploy.static.akamaitechnologies.com (23.214.208.112): icmp_seq=7 ttl=41 time=208 ms
64 bytes from a23-214-208-112.deploy.static.akamaitechnologies.com (23.214.208.112): icmp_seq=8 ttl=41 time=228 ms
64 bytes from a23-214-208-112.deploy.static.akamaitechnologies.com (23.214.208.112): icmp_seq=9 ttl=41 time=250 ms
64 bytes from a23-214-208-112.deploy.static.akamaitechnologies.com (23.214.208.112): icmp_seq=10 ttl=41 time=154 ms
64 bytes from a23-214-208-112.deploy.static.akamaitechnologies.com (23.214.208.112): icmp_seq=11 ttl=41 time=294 ms
64 bytes from a23-214-208-112.deploy.static.akamaitechnologies.com (23.214.208.112): icmp_seq=12 ttl=41 time=150 ms
64 bytes from a23-214-208-112.deploy.static.akamaitechnologies.com (23.214.208.112): icmp_seq=13 ttl=41 time=235 ms
64 bytes from a23-214-208-112.deploy.static.akamaitechnologies.com (23.214.208.112): icmp_seq=14 ttl=41 time=252 ms
64 bytes from a23-214-208-112.deploy.static.akamaitechnologies.com (23.214.208.112): icmp_seq=15 ttl=41 time=173 ms
64 bytes from a23-214-208-112.deploy.static.akamaitechnologies.com (23.214.208.112): icmp_seq=16 ttl=41 time=562 ms
64 bytes from a23-214-208-112.deploy.static.akamaitechnologies.com (23.214.208.112): icmp_seq=17 ttl=41 time=278 ms
64 bytes from a23-214-208-112.deploy.static.akamaitechnologies.com (23.214.208.112): icmp_seq=18 ttl=41 time=197 ms
64 bytes from a23-214-208-112.deploy.static.akamaitechnologies.com (23.214.208.112): icmp_seq=19 ttl=41 time=149 ms
^C
--- e2867.dsca.akamaiedge.net ping statistics ---
19 packets transmitted, 19 received, 0% packet loss, time 22475ms
rtt min/avg/max/mdev = 140.595/247.542/562.392/93.493 ms
[analyst@secOps ~]$ sudo ./lab.support.files/scripts/cyberops_extended_topo_no_fw.py
*** Adding controller
*** Add switches
*** Add hosts
*** Add links
*** Starting network
*** Configuring hosts
R1 R4 H1 H2 H3 H4 H5 H6 H7 H8 H9 H10 H11
*** Starting controllers
*** Starting switches
*** Add routes
*** Post configure switches and hosts
*** Starting CLI:
mininet> xterm R1
```

Snort was started on the router interface to monitor network traffic for malicious activity.



```
"Node: R1" (as superuser)
Using ZLIB version: 1.2.11
Rules Engine: SF_SNORT_DETECTION_ENGINE Version 3.1 <Build 1>
ld 1>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
Preprocessor Object: appid Version 1.1 <Build 5>
Commencing packet processing (pid=2317)
07/14-11:09:48.933205 fe80::549a:d3ff:fefa:1faa -> ff02::2
IPV6-ICMP TTL:255 TOS:0x0 ID:0 IpLen:40 DgmLen:56
=====
07/14-11:10:27.845371 fe80::48a4:beff:fe66:4be4 -> ff02::2
IPV6-ICMP TTL:255 TOS:0x0 ID:0 IpLen:40 DgmLen:56
=====
```

The web server running on H10 was verified using the netstat -tunpa command to confirm that the required services were active.

```

*** Adding controller
*** Adding controller

"Node: H10" (as superuser)

[root@secOps analyst]# ./lab.support.files/scripts/mal_server_start.sh
[root@secOps analyst]# netstat -tunpa
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
PID/Program name
tcp        0      0 0.0.0.0:6666             0.0.0.0:*               LISTEN
2434/nginx: master
[root@secOps analyst]#

```

A simulated malicious executable was downloaded from the server. Snort immediately detected the activity and generated an alert containing detailed information about the event.

```
"Node: R1" (as superuser)

[root@secOps analyst]# tail -f /var/log/snort/alert
07/14-11:17:41.642606  [**] [1:1000003:0] Malicious Server Hit! [**]
[Priority: 0] {TCP} 209.165.200.235:42550 -> 209.165.202.133:6666

[redacted]
[redacted]
```

To capture the network traffic for later analysis, tcpdump was started before repeating the malware download. The captured packets were saved as a .pcap file.

```

W32.Nimda.Amm 100% 404.50K 4.1 KB/s in 0.02s
2026-07-14 11:17:41 (27.7 MB/s) - 'W32.Nimda.Amm.exe' saved [475448/475448]

[root@sec0ps analyst]# tcpdump -i H5-eth0 -w nimda.download.pcap & [1] 2594
[root@sec0ps analyst]# tcpdump: listening on H5-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes

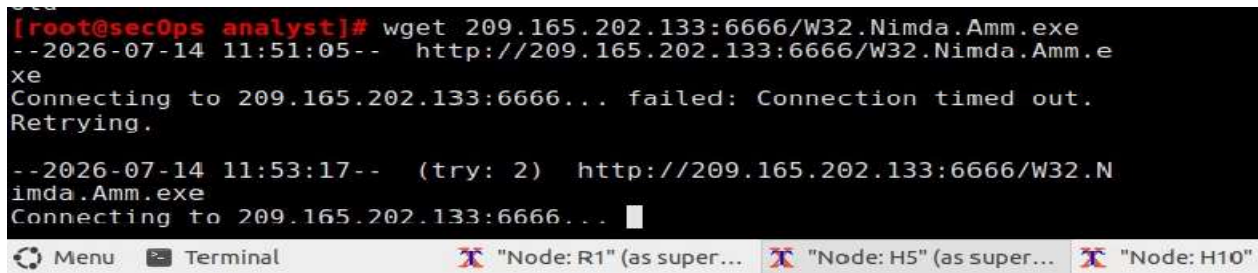
secu [root@sec0ps analyst]#

```

Existing firewall rules were reviewed before creating a new iptables rule to block access to the malicious file. After applying the rule, another download attempt was made. The request failed, confirming that the firewall rule successfully prevented access to the malicious content.

```
[root@secOps analyst]# wget 209.165.202.133:6666/W32.Nimda.Amm.exe
--2026-07-14 11:51:05-- http://209.165.202.133:6666/W32.Nimda.Amm.e
xe
Connecting to 209.165.202.133:6666... failed: Connection timed out.
Retrying.

--2026-07-14 11:53:17-- (try: 2) http://209.165.202.133:6666/W32.N
imda.Amm.exe
Connecting to 209.165.202.133:6666... █
```



5. Results

The lab objectives were successfully achieved.

Snort detected the malicious file download and generated detailed alerts describing the suspicious activity. Packet capture using tcpdump successfully recorded the network traffic for later analysis.

After implementing the firewall rule using the FORWARD chain in iptables, subsequent attempts to download the malicious file were blocked, demonstrating the effectiveness of firewall-based traffic filtering.

6. Challenges Encountered

One of the most challenging aspects of this lab was understanding how Mininet operates. Initially, it was not immediately clear why multiple terminal windows were required. As the lab progressed, I realized that each terminal represented a different network device performing a specific role. This design made it possible to simultaneously monitor alerts, simulate client activity, and manage the server, closely resembling a real enterprise network.

Another practical challenge involved transferring the packet capture (.pcap) files from the Linux virtual machine to the Windows host computer for documentation. This required troubleshooting VirtualBox file-sharing functionality before the files could be successfully copied.

Although I had previously learned about Linux firewalls, this was my first practical experience creating firewall rules using iptables, making it a valuable hands-on learning experience.

7. Lessons Learned

This lab significantly improved my understanding of how intrusion detection and firewall technologies complement each other within a layered security architecture.

I learned how to start and monitor Snort, interpret IDS alerts, capture network traffic using tcpdump, and analyze evidence of malicious activity. Most importantly, I gained practical

experience following a simplified SOC analyst workflow: detecting suspicious activity, investigating security alerts, and implementing a containment measure using firewall rules.

The lab also strengthened my understanding of the iptables INPUT, OUTPUT, and FORWARD chains, and demonstrated why the FORWARD chain is used to control traffic passing through a router.

8. Skills Demonstrated

- Network Intrusion Detection (Snort)
- Linux Firewall Administration (iptables)
- Network Traffic Monitoring
- Packet Capture (tcpdump)
- Security Alert Analysis
- Linux Command Line
- Mininet Network Simulation
- Incident Detection and Containment
- Cybersecurity Lab Documentation

9. Conclusion

This project provided valuable practical experience in detecting and preventing malicious network activity within a simulated enterprise environment. By combining Snort for intrusion detection with iptables for traffic filtering, I gained a deeper appreciation of how multiple security controls work together to protect networks.

Beyond the technical skills acquired, this lab strengthened my understanding of the investigative process followed by SOC analysts, from identifying suspicious activity and analyzing alerts to implementing effective containment measures. These practical experiences contribute to my ongoing development in network defense and cybersecurity operations.

10. References

- Cisco Networking Academy. *Network Defense Course*.
- Cisco Networking Academy. *Snort and Firewall Rules Lab*.

- Snort Documentation.
- Linux iptables Documentation.
- tcpdump Manual Pages.

Cybersecurity Portfolio Project #01

Author: Adaeze Claret Etigbue

First Class B.Sc. Computer Science Graduate | Cybersecurity Enthusiast | SOC Analyst in Training